Spring Data JPA项目，当把一个方法的执行标记成事务执行，同时设置成readOnly的模式，会有什么风险或者潜在的问题 @Transactional(readOnly = true)
public void testTransaction() {
// jdbcTemplate.execute("select ...");
System.out.println("Done");
}

在 Spring Data JPA 项目中，如果把方法标记为

Spring Framework 的 `@Transactional(readOnly = true)`，确实可以用于**只读查询优化**，但也存在一些**风险和潜在问题**。你的例子：

```
@Transactional(readOnly = true)
public void testTransaction() {
    // jdbcTemplate.execute("select ...");
    System.out.println("Done");
}
```

表面上没有问题，但在实际系统中需要注意以下几点。

一、readOnly = true 的真实含义

在 Spring Framework 中：

`readOnly = true` **不是数据库层强制只读**，而是一个**优化提示**，主要影响：

1. Hibernate Flush 策略
2. JDBC Connection readOnly flag
3. ORM 的脏检查行为

例如在使用 Hibernate 时：

- `FlushMode` 可能变成 `MANUAL`
- 脏检查减少

- 可能不会自动 flush

二、潜在风险

1 修改数据可能不会被保存（最常见问题）

如果在 readOnly 事务里修改实体：

```
@Transactional(readOnly = true)
public void updateUser() {
    User user = userRepo.findById(1).get();
    user.setName("Tom");   // 修改
}
```

在 Hibernate 中：

- **不会自动 flush**
- 修改可能 **不会被持久化**

表现为：

```
代码执行了
数据库没更新
```

而且 **不会报错**，这是非常隐蔽的问题。

---

2 某些数据库可能直接报错

如果 JDBC driver 支持 readOnly：

```Java
```

部分数据库可能拒绝写操作，例如：

- PostgreSQL
- MySQL 某些配置
- Oracle

写 SQL 可能直接报错：

```
Connection is read-only
```

---

3 JPA EntityManager 行为改变

在 readOnly 事务中：

- flush 被关闭
- dirty checking 可能被跳过
- persistence context 行为不同

可能导致：

```
逻辑依赖 flush 的代码失效
```

例如：

```java
save(entity);
```

query 可能查不到刚刚 save 的数据。

---

4 影响二级缓存或统计

在 Hibernate 中：

readOnly session 可能导致：

- entity 被标记为 readOnly
- 修改不会进入缓存更新

某些缓存策略可能出现异常行为。

---

5 事务传播带来的隐蔽问题

如果外层事务是：

```
@Transactional(readOnly = true)
```

而内部方法：

```
@Transactional
```

默认传播 `REQUIRED` 会加入外层事务。

结果：

```
内部写操作仍然在 readOnly 事务
```

导致：

- 写入失败
- 或 silently ignore

三、什么时候适合使用 readOnly

推荐只在 **纯查询方法** 使用：

```java
@Transactional(readOnly = true)
public List<User> listUsers() {
    return repo.findAll();
}
```

典型场景：

- 查询 API
- 报表
- 搜索
- 统计

优点：

- 减少 flush
- 减少脏检查

- 提高性能

四、Spring 官方建议

只在 **Service 查询方法** 使用：

```
@Service
public class UserService {

    @Transactional(readOnly = true)
    public User getUser(Long id) {}

    @Transactional
    public void updateUser(...) {}
}
```

不要在：

- 写方法
- 复杂逻辑方法
- 可能修改实体的方法

使用 readOnly。

五、你这个代码的实际影响

你的代码：

```
@Transactional(readOnly = true)
public void testTransaction() {
    System.out.println("Done");
}
```

其实：

几乎没有任何意义。

因为：

- 没有 DB 操作
- 只是打印

但仍然会：

- 开启事务
- 获取 connection

反而增加开销。

六、实际项目中最危险的场景

真实生产系统常见 bug：

```
@Transactional(readOnly = true)
public void processOrder() {
    order.setStatus(PAID);   // 修改
}
```

上线后：

```
订单状态一直没更新
但代码看起来没问题
```

排查非常困难。

七、总结

`@Transactional(readOnly = true)` 的风险主要是：

1️⃣ 修改数据可能不会 flush
2️⃣ 某些数据库禁止写入
3️⃣ JPA 行为改变（dirty checking / flush）
4️⃣ 事务传播导致写逻辑失效
5️⃣ 容易产生隐蔽 bug

原则：

```
readOnly = true 只用于纯查询方法
```